P0596
To: EWG
From: Daveed Vandevoorde (`daveed@edg.com`)
Date: 2017-02-02

`std::constexpr_trace` and `std::constexpr_assert`

Writing a `constexpr` function is not particularly difficult: We get to use the same idioms as we do with any traditional C++ functions. However, since the evaluation of those functions occurs in the environment of the compiler (or translator), we typically cannot use traditional tools to debug `constexpr` functions. We don't even have portable ways to produce "console output" to trace intermediate results of `constexpr` computation.

I therefore would like to propose to add to standard library function templates (handled "magically" by the compiler) to enable some "console output" and to abort the evaluation process when needed. The first function is declared as follows:

```
namespace std {
  template<typename ... Ts> constexpr
    void constexpr_trace(Ts && ... values);
}
```

Any evaluation of this function is a constant-expression that does not affect the abstract machine in any way. The non-normative encouragement is for the compiler/translator to report the values passed to the function in some useful way (as a kind of diagnostic), whenever called "at compile time". For example:

```
// test.cpp:
constexpr int sqr(int n) {
  if (n > 100) {
    std::constexpr_report("Largish sqr operand", n);
  }
  return n*n;
}
constexpr int n1 = sqr(128); // Some kind of output, ideally.
int x = 1000;
int n2 = sqr(x);  // No diagnostic output.
```

The intent is that no diagnostic output is produced for `n2` because the associated evaluation of `std::constexpr_report` is not "at compile time". For `n1`, a diagnostic like the following might be emitted:

```
test.cpp, line 4: constexpr_report
"Largish sqr operand" 128
```

I think it would also be useful to add a similar `std::constexpr_assert` function template:

```
namespace std {
  template<typename ... Ts> constexpr
    void constexpr_assert(bool, Ts && ... values);
}
```

Calling this template with a `false` first operand renders a program ill-formed, but a diagnostic is only required if the call occurs in a context where a constant-expression is required or—for static-lifetime initializers—preferred. There is non-normative encouragement to output the remaining operand values as part of the diagnostic in a way that is similar to what is done with `std::constexpr_trace`.